

PROJECT AND TEAM INFORMATION

Project Title

SmartSearch : Intelligent Local Document Search Engine

Student / Team Information

<i>Team Name:</i>	<i>Deus Ex Machina</i>
Team member 1 (Team Lead)	<i>Pandey, Hitarth –</i> <i>2510012711</i> <i>hitarthpandey2gmail.com</i> 
Team member 2	<i>Dobhal, Shivang –</i> <i>2510010329</i> <i>shivangdobhal99@gmail.com</i> 
Team member 3	<i>urooz, Tabinda –</i> <i>2510012283</i> <i>tabindaurooz180@gmail.com</i> 
Team member 4	<i>Sharma, Abhay –</i> <i>2510018265</i> <i>Abhay.h24fe@gmail.com</i> 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Nowadays, students store various kinds of files on their computers such as notes, PDFs, assignments, books, project files, etc. As the number of files increases, it becomes hard to find the exact information we are looking for. Sometimes, we remember the context but not the file in which it was stored. We are then forced to open various files and search for the desired information which is very time-consuming. Our project will solve this problem by providing a way to search in the user's files fast and efficiently. The idea is to build a Mini Search Engine that can search in the user's files. This mini search engine would let the user choose a folder which contains all the files they want to search in. It will parse and store information from these files. It will then allow the user to search for a keyword or phrase, and the mini search engine will return the relevant documents to the query.

The project's main objective is to allow the user to search a large amount of data quickly. The search engine can answer more queries in a shorter time than a human can look through a couple of documents.

We will be using various Data Structures and Algorithms in our project such as hash tables, tries, priority queues, and dynamic programming.

Therefore, the project is important to us as students because it will solve a real-world problem that we encounter daily. Moreover, this project will help us understand how to implement DSA and OOP concepts in a real life scenario.

State of the Art / Current solution (1 pt)

Currently, the problem is mostly solved by using the standard search options provided by computers and file managers. One can search for a file using a certain name or type a specific word to find documents containing the desired keyword. Moreover, various applications like PDF viewers or text editors sometimes have a search function, but it usually requires one to perform a search in every single file individually.

The last option currently is a manual search through one's folders and documents looking for an item containing the desired information. It is not difficult when one has only a couple of files to go through, but it becomes significantly more challenging when talking about hundreds of notes, PDFs, assignments, and other documents.

Moreover, there are also several general search tools available in the market, but they primarily rely on searching through files and not the actual contents of documents.

Our project proposes to create a local search engine that would be able to index the contents of selected documents, allowing one to quickly search through them. The search would be performed by applying one's query to all the documents simultaneously and then presenting one with the results sorted by relevance. The search engine would utilize various DSA concepts, including hashing, tries, priority queues, and others.

Project Goals and Milestones (2 pts)

The main point of our project is to create a Mini Search Engine capable of making a search in a big amount of local documents faster and more comfortable. The given system should allow the user to choose the folders with their notes, PDFs, assignments, and other documents of supported formats and search in them from a single place.

The project will be mostly focused on using the Data Structures, Algorithms, and Object-Oriented Programming approaches to address the given task efficiently. Moreover, it is important to learn how the different topics covered in the course can be integrated into the final product to create a real-life application.

Initial Milestones:

- 1-Define the general idea and functionality of the search engine.*
- 2-Set up a C++ project with OOP features.*
- 3-Design the document reading and text processing module.*
- 4-Develop an inverted index implementation with hashing to make document search efficient.*
- 5-Implement a basic search feature and show the found results.*

Further Milestones:

- 1-Implement a Trie structure to perform autocomplete and suggest search terms.*
- 2-Introduce a Priority Queue/Heap to sort the search results by their relevancy.*
- 3-Apply edit distance with dynamic programming to implement a basic spelling corrector.*
- 4-Introduce AND, OR logic and phrase search.*
- 5-Develop a search history and rescan/update module*
- 6-Develop a Qt-based GUI to add folders, search, and display results.*
- 7-Test the system with a big amount of files and compare performance with a linear search.*
- 8-Finally, assemble all the project parts, debug and prepare final documentation and presentation.*

Project Approach (3 pts)

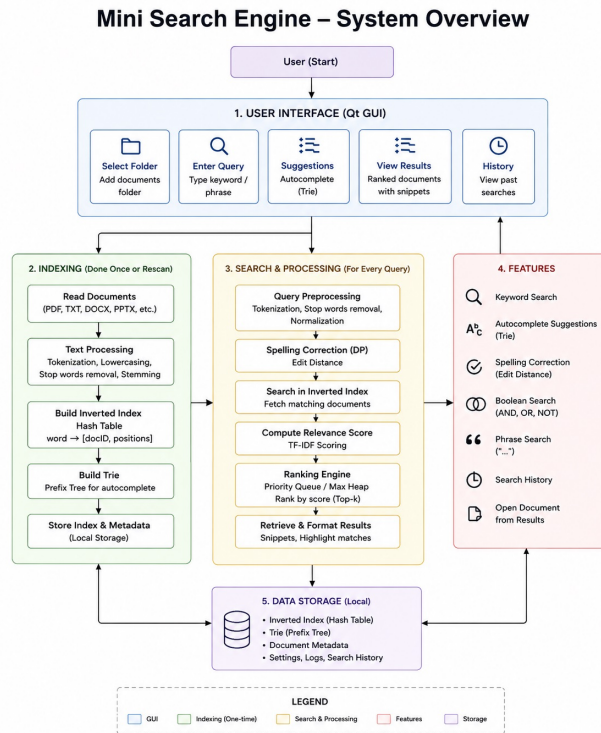
We plan to develop the project as a local document search engine, capable of searching files saved by the user on their computer. It will prioritize having an intuitive interface, while implementing Data Structures and Algorithms in the most optimal ways to make the search as efficient as possible.

The project will primarily use C++ language to demonstrate a variety of DSA and OOP principles. The application will first ask the user to select the folder, which will then be analyzed for the documents it contains. The text within the documents will be extracted, cleaned, and tokenized.

An inverted index with hash tables will be implemented, which would store the words found in the documents. We will also design a Trie for the purpose of Autocomplete. To sort the search results by relevance, we will utilize either Priority Queue or Max Heap data structures. For the task of suggesting the possible misspelled words, we plan to apply Dynamic Programming with Edit Distance. Finally, implementing Boolean and phrase searches can be done by using string and set operations. The GUI will be built with Qt C++ library and offer a desktop experience similar to the one offered by Web search engines. The functionality will include a button to add a folder, input field for the query, a display of suggestions, and a list of relevant results.

The code will be organized with the help of CMake, while Git/GitHub will provide version control. The application will initially be developed for Windows platform, but other OS'es can be supported as well. Finally, we will separate the DSA and Search implementation from the GUI to further emphasize the decoupling between the GUI and the business logic and make future development easier.

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

1-A working search engine application with a simple GUI (developed using Qt) where the user can select a folder, enter a search word, and find relevant documents.

2-An indexing module that parses the contents of files (in PDF, TXT, DOCX, and PPTX formats), tokenizes the words, and stores them in a hash table (word to document list) and a trie (for autocomplete feature).

3-A search module that finds relevant documents, autocorrects misspelled words, ranks the documents using the TF-IDF algorithm (so that the most relevant document appears on the top), and displays the top documents using a max-heap.

4-Other features include the autocomplete suggestions, boolean queries (AND/OR/NOT operators), phrase searches (using quotes), and search history.

5-The data persistence layer saves the index data so that the search engine does not have to rebuild the index every time the application is launched.

6-A report that explains the code structure, the DSA concepts (hash table, trie, max-heap, dynamic programming), and the OOP concepts used (classes and objects, encapsulation).

7-A final demo of the working search engine application that can search documents quickly.

In short, a C++ search engine application that demonstrates the concepts of DSA and OOP, along with a report and a demo.

Assumptions

- 1-All documents are stored locally on the user's machine (no internet/cloud search needed).
- 2-PDF, TXT, DOCX, PPTX file formats are supported. All other formats are out of scope.
- 3-The amount of data to be indexed is relatively small (a few hundred to a few thousand documents), not web-scale.
- 4-Only English text is searched. The application does not support searching text in other languages.
- 5-The index is rebuilt (rescanned) manually when new documents are added. There is no need for real-time indexing.
- 6-The user has only read permissions for the selected directory and its content.
- 7-The application is not distributed. It runs in the context of a single machine.

References

Books

Data Structures and Algorithms in C++ by Adam Drozdek

Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein (CLRS)

Object-Oriented Programming in C++ by Robert Lafore

Official Documentation

C++ reference: <https://en.cppreference.com/>

Qt documentation (for GUI): <https://doc.qt.io/>

STL containers (`unordered_map`, `priority_queue`): <https://cplusplus.com/reference/stl/>

Concept References

GeeksforGeeks – Trie Data Structure: <https://www.geeksforgeeks.org/trie-insert-and-search/>

GeeksforGeeks – Inverted Index: <https://www.geeksforgeeks.org/inverted-index/>

GeeksforGeeks – Edit Distance (DP): <https://www.geeksforgeeks.org/edit-distance-dp-5/>